

DSA521S DATA STRUCTURES AND ALGORITHMS 1

GROUP MINI-PROJECT 2026

NUST Service Centre Simulation: Designing and Evaluating Data Structures and Algorithms

Project Report

Student Number	Full Name
225012219	Hileni NN Uukanga
223053376	Junior Mupetami
224086391	Festus Shinedima Mt
223122386	Rebazar Allembe
221019855	Naomi kamenye
225082470	Maria N Tjoya

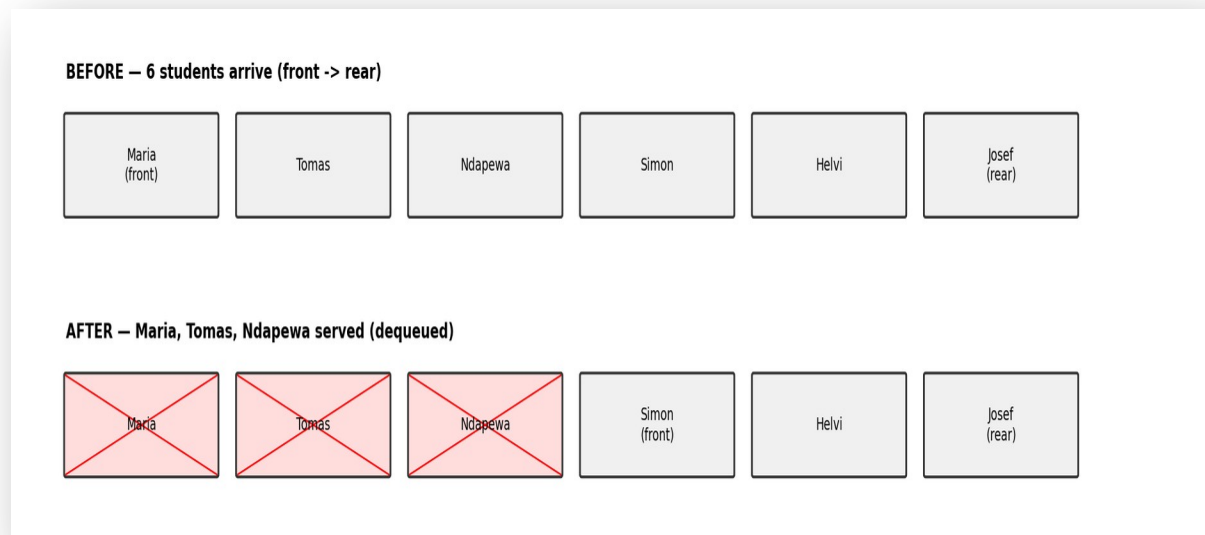
GitHub Repository:

[<https://github.com/225012219-uukanga/DSA521S-Group-Mini-Project.git>]

2. Data Structures (Part A)

A1 Waiting Line: Queue

A queue was chosen for the waiting line because students must be served strictly in the order they arrive first come, first served. A queue's First-In-First-Out (FIFO) behaviour models this directly: enqueue() adds a new arrival to the rear and dequeue() always removes the student who has been waiting longest, at the front. This matches real service-centre behaviour far better than a stack (which would serve the most recent arrival first) or an array (which would require manually shifting every element on each removal). The queue was implemented using a custom linked structure (front and rear pointers over a chain of nodes) rather than Java's built-in Queue class, as required.

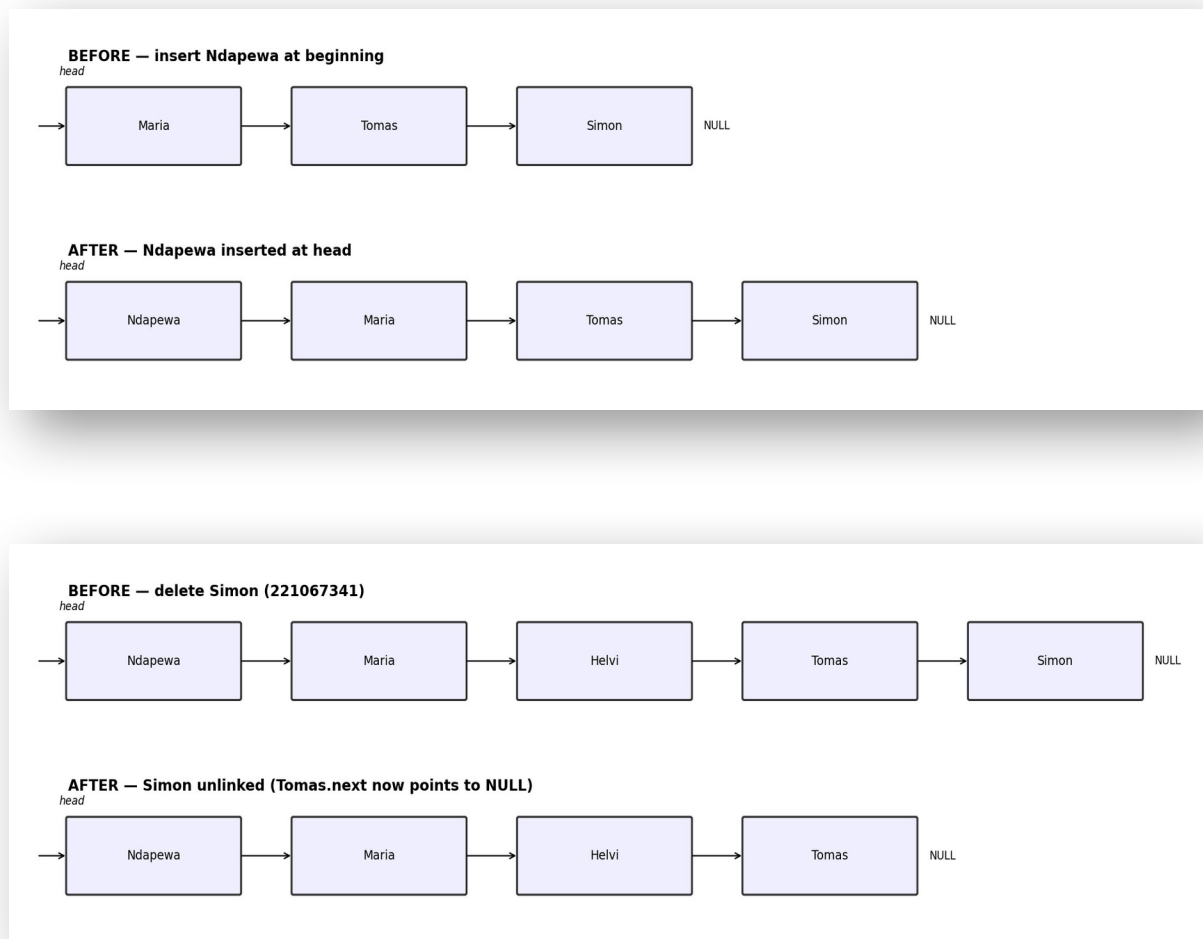


Demonstration: six students (Maria, Tomas, Ndapewa, Simon, Helvi, Josef) were enqueued in arrival order. Three students (Maria, Tomas, Ndapewa) were then dequeued and served, leaving Simon at the front of the remaining queue exactly as expected under FIFO ordering.

A2 Student Service Records: Singly Linked List

Each node in the list stores a student object (Student Number, Name, Service Type, Estimated Service Time) plus a pointer to the next node. A linked list was chosen over an array because service records are inserted and deleted frequently and at arbitrary positions (e.g. correcting a record, removing a completed record) a linked list handles this by simply re-pointing a few 'next' references, whereas an array would require shifting every subsequent element.

Below are the before/after diagrams for two link changing operations: insertion at the head, and deletion of a middle node.



Demonstration performed: insertion at the beginning, insertion at the end, insertion at a specified position (3rd), deletion of a record, searching by student number, and full traversal all shown working correctly in the program output.

A3 Postfix Expression Evaluation: Stack

A custom array-based stack (push, pop, peek no built-in java. util.Stack) evaluates postfix expressions. Numbers are pushed onto the stack; when an operator is read, the required operands are popped up, the operation applied, and the result pushed back. This is the correct structure because postfix evaluation naturally processes operand in a Last-In-First-Out order the two most recently pushed values are always the ones an operator needs.

Trace for $5\ 3\ +\ 2\ *$:

Push 5 -> Stack: [5]
Push 3 -> Stack: [5, 3]
Apply '+': $5+3=8$ -> Stack: [8]
Push 2 -> Stack: [8, 2]
Apply '*': $8*2=16$ -> Stack: [16]

Final result: 16

A4 Daily Statistics: Array

Service times of served students are stored in an array. Every statistic (total, average, highest, lowest, count over 10 minutes) is computed by manually traversing the array with a loop no built-in `max()`/`min()`/`sum()` shortcuts. An array is appropriate here because the data is a simple fixed-size collection of numbers that only needs sequential reading, not insertion/deletion at arbitrary positions.

Sample run (10 service times: 12, 5, 8, 4, 6, 10, 15, 3, 9, 11):

Total students served: 10

Total service time: 83 min

Average service time: 8.30 min

Highest service time: 15 min

Lowest service time: 3 min

Services longer than 10 min: 3

A5 Data-Structure Justification

- **Array (A4):** Chosen because the service-time data is a static, fixed collection that only needs sequential traversal for statistics arrays give fast, direct indexed access with no need for dynamic resizing or mid-list insertion.
- **Singly Linked List (A2):** Chosen because student records are added and removed at arbitrary points as the day progresses. A linked list allows insertion/deletion in $O(1)$ once the position is found, by simply updating a next pointer no shifting of other elements required.
- **Stack (A3):** Chosen because postfix evaluation is inherently a Last-In-First-Out process each operator always acts on the two most recently seen operands, which is exactly what push/pop provide.
- **Queue (A1):** Chosen because the waiting line must preserve strict arrival order (First-In-First-Out) enqueue/dequeue directly model students joining the back of the line and being served from the front.

3. Sorting Algorithms (Part B)

All four algorithms were run on the fixed array [17, 5, 23, 8, 14, 3, 11, 20, 6, 9], sorting to [3, 5, 6, 8, 9, 11, 14, 17, 20, 23].

B1 Selection Sort

Repeatedly finds the smallest remaining value and swaps it into the current position.

After pass 1: [3, 5, 23, 8, 14, 17, 11, 20, 6, 9]

After pass 2: [3, 5, 23, 8, 14, 17, 11, 20, 6, 9]

After pass 3: [3, 5, 6, 8, 14, 17, 11, 20, 23, 9]

Total comparisons: 45 Total swaps: 7

B2 Insertion Sort

Build a sorted section at the front of the array, shifting larger values right to make room for each new value.

After pass 1: [5, 17, 23, 8, 14, 3, 11, 20, 6, 9]

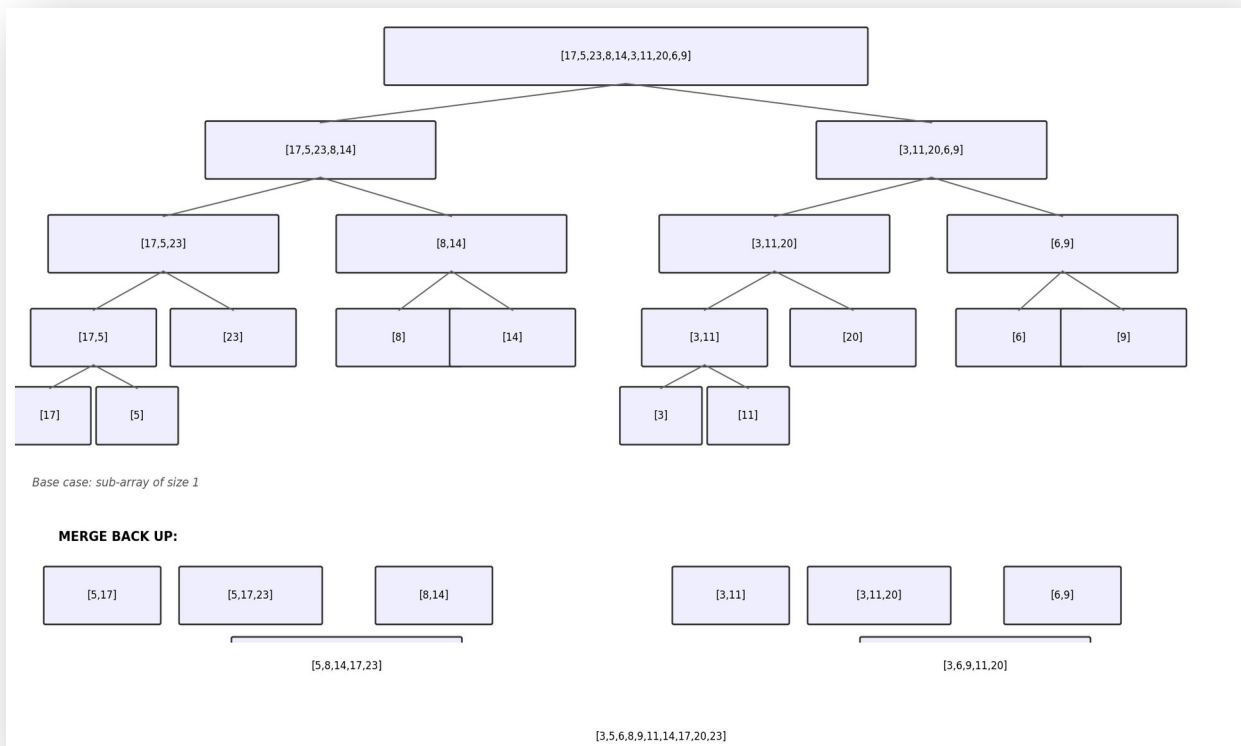
After pass 2: [5, 17, 23, 8, 14, 3, 11, 20, 6, 9]

After pass 3: [5, 8, 17, 23, 14, 3, 11, 20, 6, 9]

Total comparisons: 32 Total shifts: 25

B3 Merge Sort

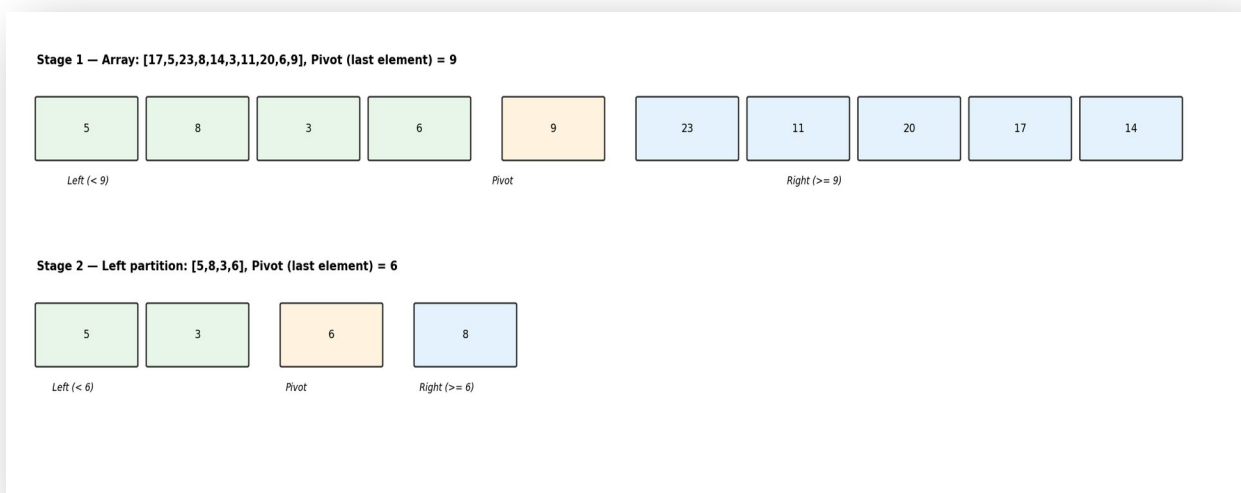
Divide the array in half repeatedly until sub-arrays of size 1 (the base case) remain, then merge pairs back together in sorted order.



Total comparisons: 23

B4 Quick Sort

Pivot selection rule: the last element of each sub-array (Lomuto partition scheme). Elements smaller than the pivot are moved left, larger ones right, then the pivot is placed between the two partitions.



Total comparisons: 20 Total swaps: 15

4. Algorithm Experiment (Part C)

Each algorithm was given an identical copy of the same randomly generated array for each input size, and only the sorting call itself was timed (System.nanoTime() immediately before/after). Only data-value comparisons were counted.

Algorithm	Input Size	Comparisons	Time (ns)
Selection Sort	20	190	11854
Insertion Sort	20	102	24350
Merge Sort	20	60	18543
Quick Sort	20	83	17202
Selection Sort	50	1225	37694
Insertion Sort	50	607	26356
Merge Sort	50	223	21267
Quick Sort	50	275	17344
Selection Sort	100	4950	131617
Insertion Sort	100	2321	100910
Merge Sort	100	540	44343
Quick Sort	100	635	35932
Selection Sort	500	124750	4679374
Insertion Sort	500	62300	2634258
Merge Sort	500	3877	293186
Quick Sort	500	4763	1407647

Almost Sorted Test (100 elements, 5 neighbouring pairs swapped)

Algorithm	Comparisons	Time (ns)
Selection Sort	4950	67909
Insertion Sort	104	5551
Merge Sort	357	6480
Quick Sort	4735	17310

Analysis Questions

1. Which algorithm performed the fewest comparisons?

Merge Sort consistently used the fewest comparisons across all input sizes (e.g. 3,877 vs Selection Sort's 124,750 at size 500), matching its $O(n \log n)$ behaviour.

2. Which algorithm performed the most comparisons?

Selection Sort performed the most comparisons at every size, because it always scans the entire remaining unsorted portion on every pass regardless of how sorted the data already is – an $O(n^2)$ algorithm with no best-case improvement.

3. Which algorithm performed best on the almost-sorted array?

Insertion Sort performed best on the almost-sorted array (only 104 comparisons, 5,551 ns), since most elements were already in position. Insertion Sort's inner loop exits almost immediately when data is nearly sorted, giving close to $O(n)$ performance in this case.

4. Which algorithm appeared to scale best as the input became larger?

Merge Sort and Quick Sort scaled best, with comparison counts growing roughly in proportion to $n \log n$ rather than n^2 , confirming the theoretical complexity difference from Selection/Insertion Sort as input size increased.

5. Did the experimental results generally agree with theoretical time complexity?

Largely yes. Selection Sort and Insertion Sort's comparison counts grew quadratically with input size, matching $O(n^2)$, while Merge Sort and Quick Sort grew much more slowly, matching $O(n \log n)$. Execution times followed the same broad pattern, though with some noise from JVM scheduling and timing overhead at small sizes.

6. Is execution time alone enough to decide which algorithm is better?

No. Execution time can be affected by external factors such as JVM warm up, system load, and measurement overhead, which is why comparisons/operations counts (a hardware-independent measure) were also tracked. A fuller comparison also considers memory usage (Merge Sort needs

$O(n)$ extra space, unlike Quick Sort's in-place partitioning) and worst-case behaviour (Quick Sort can degrade to $O(n^2)$ on already sorted input with a poor pivot choice), not just a single run's timing.

5. Integrated System (Part D)

The Queue, Singly Linked List, Array statistics, and Sorting components were combined into a single menu driven console program (ServiceCentreApp.java). The postfix Stack (A3) remains a separate, standalone exercise as specified.

Menu Option	Structure / Operation Used
1	Queue — enqueue
2	Queue — dequeue
3	Queue — traversal / display
4	Singly Linked List — insertion
5	Singly Linked List — traversal / display
6	Singly Linked List — search
7	Singly Linked List — deletion
8	Array processing (statistics)
9	Sorting algorithm(s)
10	Sorting experiment
11	Exit

Screenshot 1

Program running with preloaded sample data, showing the main menu:

```

C:\Users\Hileni\Desktop\DSA521S_GroupXX_Project2026\src>java ServiceCent
reApp
Enqueued: Maria (Registration)
Enqueued: Tomas (Student Card)
Enqueued: Ndapewa (Fees)
Inserted at beginning: Maria
Inserted at position 2: Tomas
=====
CAMPUS SERVICE CENTRE
=====
1. Add student to waiting queue
2. Serve next student (remove from queue)
3. Display waiting students
4. Add student service record (Linked List)
5. Display student service records
6. Search for student record
7. Remove student record
8. Display daily statistics
9. Sort service times
10. Run sorting experiment
11. Exit
Select option: |

```

Screenshot 2

Option 3: displaying the waiting queue:

```

Select option: 3
---- Waiting Queue (front to back) ----
1. 221045678 | Maria | Registration | 12 min
2. 222034512 | Tomas | Student Card | 5 min
3. 223041876 | Ndapewa | Fees | 8 min
-----
=====
CAMPUS SERVICE CENTRE
=====
1. Add student to waiting queue
2. Serve next student (remove from queue)
3. Display waiting students
4. Add student service record (Linked List)
5. Display student service records
6. Search for student record
7. Remove student record
8. Display daily statistics
9. Sort service times
10. Run sorting experiment
11. Exit
Select option: |

```

Screenshot 3

Option 5: displaying student service records from the Singly Linked List:

```
11. Exit
Select option: 5
---- Student Service Records ----
1. 221045678 | Maria | Registration | 12 min
2. 222034512 | Tomas | Student Card | 5 min
=====

=====
CAMPUS SERVICE CENTRE
=====

1. Add student to waiting queue
2. Serve next student (remove from queue)
3. Display waiting students
4. Add student service record (Linked List)
5. Display student service records
6. Search for student record
7. Remove student record
8. Display daily statistics
9. Sort service times
10. Run sorting experiment
11. Exit
Select option: |
```

Screenshot 4

Option 9: sorting service times using Merge Sort, showing the full divide/merge trace and final comparison count:

```
Command Prompt - java Sen  X + v
Select option: 9
Choose algorithm: 1) Selection 2) Insertion 3) Merge 4) Quick
Choice: 3
Dividing [17, 5, 23, 8, 14, 3, 11, 20, 6, 9] into [17, 5, 23, 8, 14] and [3, 11, 20, 6, 9]
Dividing [17, 5, 23, 8, 14] into [17, 5, 23] and [8, 14]
Dividing [17, 5, 23] into [17, 5] and [23]
Dividing [17, 5] into [17] and [5]
Base case reached: [17]
Base case reached: [5]
Merged into: [5, 17]
Base case reached: [23]
Merged into: [5, 17, 23]
Dividing [8, 14] into [8] and [14]
Base case reached: [8]
Base case reached: [14]
Merged into: [8, 14]
Merged into: [5, 8, 14, 17, 23]
Dividing [3, 11, 20, 6, 9] into [3, 11, 20] and [6, 9]
Dividing [3, 11, 20] into [3, 11] and [20]
Dividing [3, 11] into [3] and [11]
Base case reached: [3]
Base case reached: [11]
Merged into: [3, 11]
Base case reached: [20]
Merged into: [3, 11, 20]
Dividing [6, 9] into [6] and [9]
Base case reached: [6]
Base case reached: [9]
Merged into: [6, 9]
Merged into: [3, 6, 9, 11, 20]
Merged into: [3, 5, 6, 8, 9, 11, 14, 17, 20, 23]
Sorted result: [3, 5, 6, 8, 9, 11, 14, 17, 20, 23]
Comparisons: 23
```

6. Pseudocode (Part E)

Pseudocode for all required operations, corresponding directly to the submitted Java implementation:

Queue

```
PROCEDURE enqueue(student)
    newNode <- create Node(student)
    IF rear IS NULL THEN
        front <- newNode
        rear <- newNode
    ELSE
        rear.next <- newNode
        rear <- newNode
    END IF
    size <- size + 1
END PROCEDURE
```

```

PROCEDURE dequeue()
  IF front IS NULL THEN
    PRINT "Queue is empty"
    RETURN NULL
  END IF
  served <- front
  front <- front.next
  IF front IS NULL THEN
    rear <- NULL
  END IF
  size <- size - 1
  RETURN served
END PROCEDURE

```

Stack (used for push, pop, and postfix evaluation)

```

PROCEDURE push(value)
  IF top = capacity - 1 THEN
    PRINT "Stack overflow"
  ELSE
    top <- top + 1
    items[top] <- value
  END IF
END PROCEDURE

```

```

PROCEDURE pop()
  IF top = -1 THEN
    PRINT "Stack underflow"
  ELSE
    value <- items[top]
    top <- top - 1
    RETURN value
  END IF
END PROCEDURE

```

```

PROCEDURE evaluatePostfix(expression)
  FOR EACH token IN expression
    IF token IS a number THEN
      push(token)
    ELSE // token is an operator
      b <- pop()
      a <- pop()
      result <- applyOperator(a, b, token)
      push(result)
    END IF
  END FOR
  RETURN pop() // final answer is the only value left on the
stack

```

END PROCEDURE

Singly Linked List

```
PROCEDURE insertNode(student, position)
    newNode <- create Node(student)
    IF position <= 1 OR head IS NULL THEN
        newNode.next <- head
        head <- newNode
        RETURN
    END IF
    current <- head
    index <- 1
    WHILE index < position - 1 AND current.next IS NOT NULL
        current <- current.next
        index <- index + 1
    END WHILE
    newNode.next <- current.next
    current.next <- newNode
END PROCEDURE
```

```
PROCEDURE deleteNode(studentNo)
    IF head IS NULL THEN
        RETURN false
    END IF
    IF head.data.studentNo = studentNo THEN
        head <- head.next
        RETURN true
    END IF
    current <- head
    WHILE current.next IS NOT NULL AND
current.next.data.studentNo != studentNo
        current <- current.next
    END WHILE
    IF current.next IS NULL THEN
        RETURN false    // not found
    END IF
    current.next <- current.next.next
    RETURN true
END PROCEDURE
```

```
PROCEDURE searchNode(studentNo)
    current <- head
    WHILE current IS NOT NULL
        IF current.data.studentNo = studentNo THEN
            RETURN current.data
        END IF
        current <- current.next
    END WHILE
```

```

        END WHILE
        RETURN NULL    // not found
    END PROCEDURE

PROCEDURE traverseList()
    current <- head
    WHILE current IS NOT NULL
        PRINT current.data
        current <- current.next
    END WHILE
END PROCEDURE

```

Sorting Algorithms

```

PROCEDURE selectionSort(arr)
    n <- length(arr)
    FOR i <- 0 TO n - 2
        minIndex <- i
        FOR j <- i + 1 TO n - 1
            IF arr[j] < arr[minIndex] THEN
                minIndex <- j
            END IF
        END FOR
        IF minIndex != i THEN
            SWAP arr[i] AND arr[minIndex]
        END IF
    END FOR
END PROCEDURE

```

```

PROCEDURE insertionSort(arr)
    n <- length(arr)
    FOR i <- 1 TO n - 1
        key <- arr[i]
        j <- i - 1
        WHILE j >= 0 AND arr[j] > key
            arr[j + 1] <- arr[j]
            j <- j - 1
        END WHILE
        arr[j + 1] <- key
    END FOR
END PROCEDURE

```

```

PROCEDURE mergeSort(arr, left, right)
    IF left >= right THEN
        RETURN    // base case: 0 or 1 element is already sorted
    END IF
    mid <- (left + right) / 2

```

```

mergeSort(arr, left, mid)
mergeSort(arr, mid + 1, right)
merge(arr, left, mid, right)
END PROCEDURE

PROCEDURE merge(arr, left, mid, right)
  copy arr[left..right] into temp[left..right]
  i <- left, j <- mid + 1, k <- left
  WHILE i <= mid AND j <= right
    IF temp[i] <= temp[j] THEN
      arr[k] <- temp[i]; i <- i + 1
    ELSE
      arr[k] <- temp[j]; j <- j + 1
    END IF
    k <- k + 1
  END WHILE
  COPY remaining elements of temp[i..mid] and temp[j..right]
into arr
END PROCEDURE

PROCEDURE quickSort(arr, low, high)
  IF low < high THEN
    pivotIndex <- partition(arr, low, high)
    quickSort(arr, low, pivotIndex - 1)
    quickSort(arr, pivotIndex + 1, high)
  END IF
END PROCEDURE

PROCEDURE partition(arr, low, high)
  pivot <- arr[high] // last element chosen as pivot
  i <- low - 1
  FOR j <- low TO high - 1
    IF arr[j] < pivot THEN
      i <- i + 1
      SWAP arr[i] AND arr[j]
    END IF
  END FOR
  SWAP arr[i + 1] AND arr[high]
  RETURN i + 1
END PROCEDURE

```

Conclusion

Part A implemented and justified each core structure individually: a linked-list-based Queue modelled the This project brought together four core data

structures and one full performance study to build a working Campus Service Centre simulation.

FIFO waiting line, a Singly Linked List managed student service records with insertion, deletion, search and traversal, a Stack evaluated postfix arithmetic expressions independently of the service-centre menu, and an Array was traversed manually to compute daily service statistics. Each structure was chosen deliberately for how its behaviour matched its task, rather than for convenience.

Part B implemented all four required sorting algorithms (Selection, Insertion, Merge and Quick Sort) from scratch on a fixed test array, with traces confirming each algorithm's internal logic the pass-by-pass swaps of Selection and Insertion Sort, the divide-and-merge recursion of Merge Sort, and the pivot-based partitioning of Quick Sort.

Part C extended this into an experimental comparison across input sizes of 20, 50, 100 and 500 elements, plus an almost-sorted case. The results confirmed the expected theoretical complexities: Selection and Insertion Sort scaled quadratically ($O(n^2)$), while Merge and Quick Sort scaled far more efficiently ($O(n \log n)$) as input size grew, with Insertion Sort performing best of all on the almost-sorted array due to its adaptive best-case behaviour.

Part D integrated the Queue, Linked List, Array statistics and Sorting components into a single menu-driven program, demonstrating that each data structure could operate correctly both independently and as part of a larger system, while the postfix Stack remained a self-contained exercise as specified.

Overall, the project demonstrates that structure selection is not arbitrary: each data structure's strengths (FIFO ordering, flexible insertion/deletion, LIFO evaluation, or direct indexed access) map directly onto the requirements of the task it was assigned, and the sorting experiment provides empirical evidence to support the theoretical time-complexity reasoning taught in the course.

7. GitHub Repository

Repository link: [<https://github.com/225012219-uukanga/DSA521S-Group-Mini-Project.git>]

Commit history demonstrates progressive development of the Queue, Linked List, Stack, Array statistics, sorting algorithms, experiment harness, and integrated system across the project period, with descriptive commit messages for each component.